

Idea: allow waiting ONLY before conflict points to reduce grounding.

30.05.25

What am I to do now?

First of all, put myself back into the context. Then, I need to understand Kep's positional encoding.

To do this all, let's return to the **simplest one-train environments**.

Running on the 'straight line': **works**. In terms of transition atoms, the answer is:

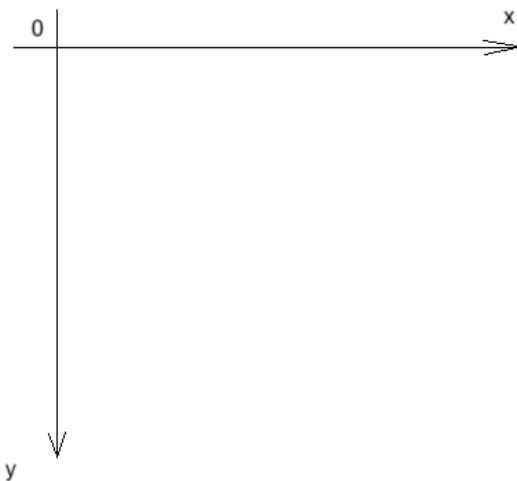
```
transition(1,0,0,north,move_forward,3,north)
transition(1,1,3,north,move_forward,6,north)
```

Yes, and in terms of 'action' atoms it is:

```
action(1,0,move_forward) action(1,1,move_forward)
```

Yes, all is correct.

Now, let's understand Kep's positional schema (and it's *still a question* whether we need to change anything at all):



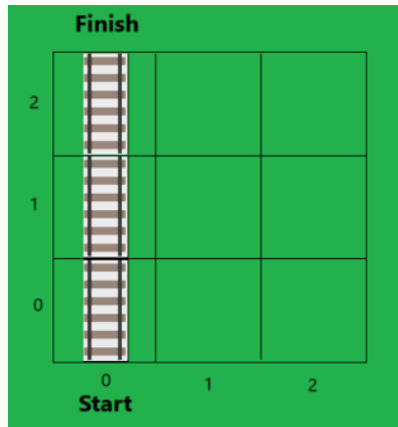
This is the way Kep encodes positions. How would this change the simplest case - the 'straight line' environment?

The horizontal axis **remains the same**.

The vertical axis is **turned upside down**.

So, yes, it must bring changes to the way we encode the environment.

Let's look at the original encoding of 'straight line':



```

train(1).
start(1, (0, 0), 0, north).
end(1, (2, 0), 2).

cell((0, 0), 32800).
cell((0, 1), 0).
cell((0, 2), 0).
cell((1, 0), 32800).
cell((1, 1), 0).
cell((1, 2), 0).
cell((2, 0), 32800).

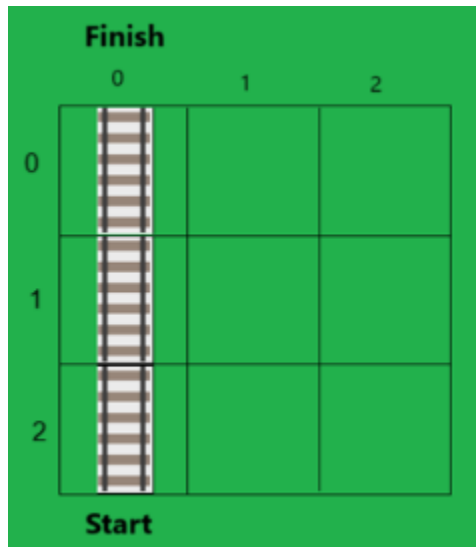
```

```

cell((2, 1), 0).
cell((2, 2), 0).

```

Let's try to rewrite it for Kep's encoding, with the vertical axis turned upside down (the arrival time won't be affected, as well as the start direction, but the start and the end positions will):



```

train(1).
start(1, (2, 0), 0, north).
end(1, (0, 0), 2).

cell((0, 0), 32800).
cell((0, 1), 0).
cell((0, 2), 0).
cell((1, 0), 32800).
cell((1, 1), 0).
cell((1, 2), 0).
cell((2, 0), 32800).
cell((2, 1), 0).
cell((2, 2), 0).

```

It seems that except for the start and the end, nothing changes *for this encoding*. Only the start and the end are literally *swapped*. EXPECTATION: It won't be that simple for even slightly more complex encodings.

EXPECTATION: a. it will fail when run now, because the 'path' atoms are described 'upside down' OR b. it will run, but only because it's the simplest case OR c. it will run, but show wrong directions ('south' instead of 'north'). C. is the most likely.

Checking: a was correct, not c. Why? Because the train can't move from the start anywhere - the direction being set to 'north' from the new starting cell violates the requirement for the 'path'

to continue. Hence, changing the way 'path' atoms are formed is mandatory even for the simplest environments.

Good. I need to rewrite all the 'path' atoms - as well as my old environments. It's a lot of work and reminds me of a simple truth: **ALWAYS READ THE INPUT TO THE TASK WITH EXTREME ATTENTION**. I could have saved a day of work there.

Starting with the simplest 'path' atoms - straight lines etc. But first, I need to visualize the new 1d encoding (it will start from another angle) and perhaps rewrite the deltas table (copy it, since the old one is perfectly checked now and is of huge value).

IDEA: **discover a function** where the input is the old encoding the output is the new one, then implement it as a Python script that will rewrite both / either the path atoms and th

LLM (Gemini): *'Discovering a function, in mathematics, means identifying a relationship where each input (x-value) corresponds to exactly one output (y-value). This can be done by examining a graph, a set of ordered pairs, or an equation. A function is a special type of relation.'*

A set of ordered pairs is exactly my case.

IDEA: if I fail to do it myself quickly enough, I can feed the ordered pairs to an LLM to discover the function.

Now, I must do a lot of hand drawing to understand these new relationships:

1. Surprisingly, the 2d to 1d positional converter **doesn't need to be changed** to handle new coordinates. I've tested the way 2d coordinates are flattened for the old and the new 'systems' and the output 1d coordinates were the same regardless of the input.

EXPECTATION: normal rules creating 'path' atoms do need changes, as well as the transition table that describes them. Checking: yes, they need to be changed, but it's simple.

2. The way the 'path' creating normal rules need to be changed can be described by a very simple function:

$$f(\Delta_{\text{old}}) = \Delta_{\text{new}}$$

That:

- a. **Flips the sign** of change if the delta old was directed north-south
- b. **Does nothing** if the delta old was directed east-west

I could write a Python script that would do this for me, but it seems more economical (due to the small number of lines to change and for more control) to just do it by hand. In the transition table, I will do it using the 'find and replace' function: ah no, I can't since it will lead to confusion immediately due to the presence of both opposite signs. Then, only by hand: **done** (almost certainly with mistakes).

Converting the transition table could be borderline useless, since path atom rules are just as explicit and require just the same transformation. Let's do it: **done**.

Let's now test, starting again from very simple environments (I must remember that *all my environments need to be reworked too if I want to run on them*):

Env	Answer	Correctness
Straight line	<pre>transition(1,0,6,north,move_forward,3,north) transition(1,1,3,north,move_forward,0,north)</pre>	Yes
Γ-turn	<pre>transition(1,0,6,north,move_forward,3,north) transition(1,1,3,north,move_forward,0,north) transition(1,2,0,north,move_forward,1,east) transition(1,3,1,east,move_forward,2,east)</pre>	Yes
Π-turn	<pre>transition(1,0,6,north,move_forward,3,north) transition(1,1,3,north,move_forward,0,north) transition(1,2,0,north,move_forward,1,east) transition(1,3,1,east,move_forward,2,east) transition(1,4,2,east,move_forward,5,south) transition(1,5,5,south,move_forward,8,south)</pre>	Yes

I need to either learn a function that will convert old encodings (there's definitely one) to new coordinates or forget about them and proceed to Kep's immediately (on the assumption that if I didn't change anything except for swapping vertical movements, then everything must work just as well as it used to) - otherwise it's **too time-consuming**. **Success** so far.

Learning a function to map all old environments to the new coordinate system seems not such a challenging task:

$$f(y_{\text{old}}) = y_{\text{new}}$$

In Python must be:

1. Input: an array of 'cell((Y, X)', TYPE) strings (or other suitable objects).
2. Extract only Y and sort in ascending order.
3. Copy the array and reverse it.
4. Zip the original and the reversed copy into one dictionary.

5. Create a new array of 'cell' strings where each Y_{new} is simply the value stored under the Y_{old} key.

Let's implement: **done.**

```
input_env_listed_cleared = [x.replace('\n', '') for x in input_env.split('.')[::-1]]

y_old = sorted(set([x[6] for x in input_env_listed_cleared]))
y_new = deepcopy(y_old)
y_new.reverse()

y_mapper = dict(zip(y_old, y_new))

def y_replacer(cell_string):
    cell_string_listed = list(cell_string)
    y_old = cell_string_listed[6]
    y_new = y_mapper[y_old]
    cell_string_listed[6] = y_new
    cell_string_new = "".join(cell_string_listed)
    return cell_string_new

output_env = [y_replacer(x) for x in input_env_listed_cleared]
```

Now I can convert any of my old environments into the new coordinate system (there still might be some bugs).

Now I keep testing the new encoding on the converted simple environments:

Env	Answer	Correctness
bagel	Answer: 1 transition(1,0,5,east,move_forward,6,east) transition(1,1,6,east,turn_left,1,north) transition(1,2,1,north,move_forward,2,east) transition(1,3,2,east,move_forward,3,east) transition(1,4,3,east,move_forward,8,south) transition(1,5,8,south,move_forward,9,west) Answer: 2 transition(1,0,5,east,move_forward,6,east) transition(1,1,6,east,turn_right,11,south) transition(1,2,11,south,move_forward,12,east) transition(1,3,12,east,move_forward,13,east) transition(1,4,13,east,move_forward,8,north) transition(1,5,8,north,move_forward,9,west)	Yes
switch trap (different ends)	transition(1,0,3,east,move_forward,4,east) transition(1,1,4,east,turn_right,7,south) transition(1,2,7,south,move_forward,8,east)	Yes
Many turns	...	Yes, Yes, Yes, Yes

31.05.25 bowel	<pre> transition(2,0,8,east,move_forward,9,east) transition(2,1,9,east,move_forward,10,east) transition(2,2,10,east,move_forward,14,south) transition(2,3,14,south,move_forward,15,east) </pre>	No. H1: smth wrong with either of the trains separately; T1: runs T2: fails Debugging: the route seems correct; the time seems correct. The starting direction seems correct. Still don't see the problem. Let's try the old coordinate encoding in the old motion encoding - if it works, then something's wrong with how the track is converted; if it doesn't, then the track was initially incorrect (H: wrong path descriptions). Yes, the old one doesn't run either. So, the problem is likely in the encoding. Let's debug by backtracking to shorter paths. DAMN! I've just confused 'east' and 'west' again.
caduceus	<pre> transition(1,0,1,south,move_forward,4,south) transition(1,1,4,south,move_forward,7,south) transition(1,2,7,south,move_forward,10,south) transition(1,3,10,south,move_forward,13,south) transition(1,4,13,south,move_forward,16,south) transition(1,5,16,south,move_forward,19,south) transition(2,0,0,east,move_forward,3,south) transition(2,1,3,south,move_forward,4,east) </pre>	Yes

	transition(2,2,4,east,move_forward,5,east) transition(2,3,5,east,move_forward,8,south) transition(2,4,8,south,move_forward,11,south) transition(2,5,11,south,move_forward,10,west) transition(2,6,10,west,move_forward,9,west) transition(2,7,9,west,move_forward,12,south) transition(2,8,12,south,move_forward,15,south) transition(2,9,15,south,move_forward,16,east) transition(2,10,16,east,move_forward,17,east) transition(2,11,17,east,move_forward,20,south)	
single_switch	transition(1,0,0,east,move_forward,1,east) transition(1,1,1,east,move_forward,2,east) transition(1,2,2,east,move_forward,8,south) transition(1,3,8,south,move_forward,14,south) transition(1,4,14,south,move_forward,13,west) transition(1,5,13,west,move_forward,12,west) transition(2,0,12,east,move_forward,13,east) transition(2,1,13,east,move_forward,14,east) transition(2,2,14,east,move_forward,15,east) transition(2,3,15,east,move_forward,16,east) transition(2,4,16,east,move_forward,17,east)	Yes
ten_by_ten	...	Yes, Yes (Solving: 10.88s 1st Model: 10.27s) Yes (two trains, surprisingly MUCH FAST than for 1 train - interesting why; need to discuss on Monday).

CONCERN: it's not very fast even on a relatively small 10x10 environment with just one train. Maybe my whole idea of its being more efficient than just explicit generation is wrong. This will be revealed only after fair testing.

Let's keep complicating the ten by ten in the new coordinates. It's interesting to observe the performance with relation to the number of trains (**I'd plot this** after enough experiments - same for **comparison testing**). Let's add one more train (let's just take the previous 2 train version and add one more train instead of planning anew). First let's add the third train without any opportunity for clashes. **Done**. It's still very fast (unlike for that single train somehow).

Now, let's introduce a crash opportunity that's not likely to happen without any waiting. **Done**.

Let's now make the overlap a little more dangerous. **Done**.

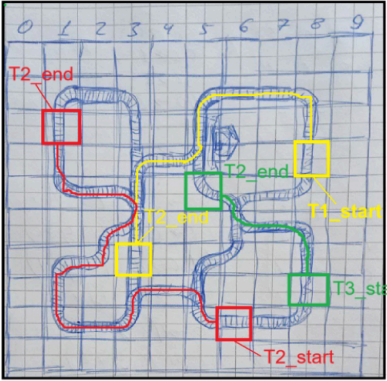
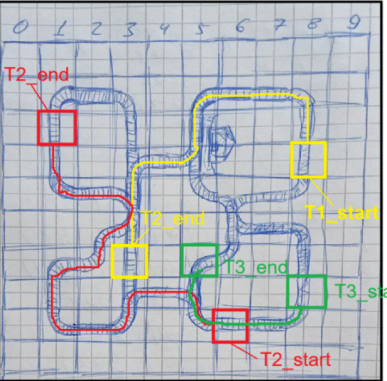
Let's now introduce the case where waiting is a must. We can do this without changing the environment - by making one of the trains start later so that it reaches the intersection at the same time as the counterpart. First, **let's clash them deliberately**. Strage, I can't crash them.

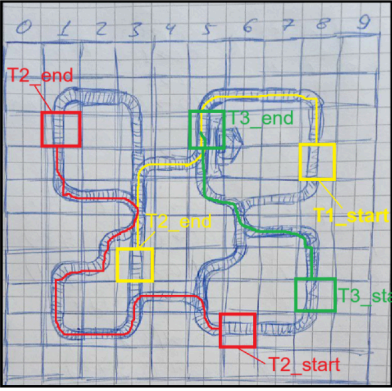
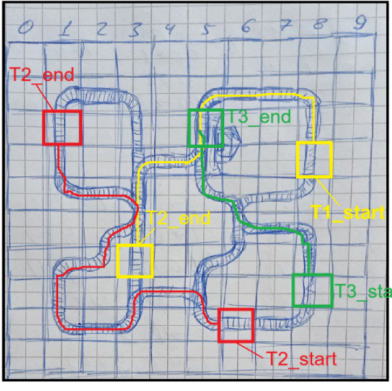
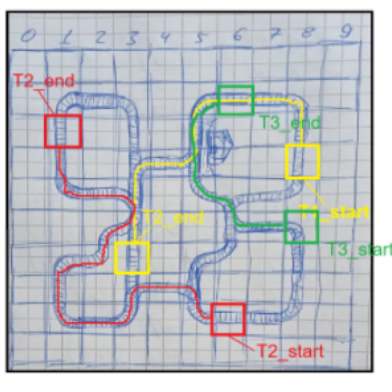
Wait actions appear, although they aren't supposed to. Let's crash them by changing the environment then.

Trying to build an environment that would crash. Let's run this single additional train first.

OBSERVATION: in my encoding, the train must be directed in a certain way at the start.

Now, the 3rd train runs well. We can try to crash it.

Image	Answer	Speed
	<pre>transition(0,0,38,north,move_forward,28,north) transition(0,1,28,north,move_forward,18,north) transition(0,2,18,north,move_forward,17,west) transition(0,3,17,west,move_forward,16,west) transition(0,4,16,west,move_forward,15,west) transition(0,5,15,west,move_forward,25,south) transition(0,6,25,south,move_forward,35,south) transition(0,7,35,south,turn_right,34,west) transition(0,8,34,west,move_forward,33,west) transition(0,9,33,west,move_forward,43,south) transition(0,10,43,south,move_forward,53,south) transition(0,11,53,south,move_forward,63,south) transition(1,0,86,west,move_forward,85,west) transition(1,1,85,west,move_forward,75,north) transition(1,2,75,north,turn_left,74,west) transition(1,3,74,west,move_forward,73,west) transition(1,4,73,west,move_forward,83,south) transition(1,5,83,south,move_forward,82,west) transition(1,6,82,west,move_forward,81,west) transition(1,7,81,west,move_forward,71,north) transition(1,8,71,north,move_forward,61,north) transition(1,9,61,north,move_forward,62,east) transition(1,10,62,east,move_forward,52,north) transition(1,11,52,north,move_forward,53,east) transition(1,12,53,east,move_forward,43,north) transition(1,13,43,north,turn_left,42,west) transition(1,14,42,west,move_forward,41,west) transition(1,15,41,west,move_forward,31,north) transition(1,16,31,north,move_forward,21,north) transition(2,0,78,north,move_forward,68,north) transition(2,1,68,north,move_forward,58,north) transition(2,2,58,north,move_forward,57,west) transition(2,3,57,west,move_forward,56,west) transition(2,4,56,west,move_forward,46,north) transition(2,5,46,north,turn_left,45,west)</pre>	(Solving: 1.45s 1st Model: 0.31s Unsat: 1.14s)
	<pre>transition(0,0,38,north,move_forward,28,north) transition(0,1,28,north,move_forward,18,north) transition(0,2,18,north,move_forward,17,west) transition(0,3,17,west,move_forward,16,west) transition(0,4,16,west,move_forward,15,west) transition(0,5,15,west,move_forward,25,south) transition(0,6,25,south,move_forward,35,south) transition(0,7,35,south,turn_right,34,west) transition(0,8,34,west,move_forward,33,west) transition(0,9,33,west,move_forward,43,south) transition(0,10,43,south,move_forward,53,south) transition(0,11,53,south,move_forward,63,south) transition(1,0,86,west,move_forward,85,west) transition(1,1,85,west,move_forward,75,north) transition(1,2,75,north,turn_left,74,west) transition(1,3,74,west,move_forward,73,west) transition(1,4,73,west,move_forward,83,south) transition(1,5,83,south,move_forward,82,west) transition(1,6,82,west,move_forward,81,west) transition(1,7,81,west,move_forward,71,north) transition(1,8,71,north,move_forward,61,north) transition(1,9,61,north,move_forward,62,east) transition(1,10,62,east,move_forward,52,north) transition(1,11,52,north,move_forward,53,east) transition(1,12,53,east,move_forward,43,north) transition(1,13,43,north,turn_left,42,west) transition(1,14,42,west,move_forward,41,west) transition(1,15,41,west,move_forward,31,north) transition(1,16,31,north,move_forward,21,north) transition(2,0,68,south,move_forward,78,south) transition(2,1,78,south,move_forward,88,south) transition(2,2,88,south,move_forward,87,west) transition(2,3,87,west,move_forward,86,west) transition(2,4,86,west,move_forward,85,west) transition(2,5,85,west,move_forward,75,north) transition(2,6,75,north,move_forward,65,north)</pre>	(Solving: 1.15s 1st Model: 0.73s Unsat: 0.42s)

	<pre> transition(0,0,38,north,move_forward,28,north) transition(0,1,28,north,move_forward,18,north) transition(0,2,18,north,move_forward,17,west) transition(0,3,17,west,move_forward,16,west) transition(0,4,16,west,move_forward,15,west) transition(0,5,15,west,move_forward,25,south) transition(0,6,25,south,move_forward,35,south) transition(0,7,35,south,turn_right,34,west) transition(0,8,34,west,move_forward,33,west) transition(0,9,33,west,move_forward,43,south) transition(0,10,43,south,move_forward,53,south) transition(0,11,53,south,move_forward,63,south) transition(1,0,86,west,move_forward,85,west) transition(1,1,85,west,move_forward,75,north) transition(1,2,75,north,turn_left,74,west) transition(1,3,74,west,move_forward,73,west) transition(1,4,73,west,move_forward,83,south) transition(1,5,83,south,move_forward,82,west) transition(1,6,82,west,move_forward,81,west) transition(1,7,81,west,move_forward,71,north) transition(1,8,71,north,move_forward,61,north) transition(1,9,61,north,move_forward,62,east) transition(1,10,62,east,move_forward,52,north) transition(1,11,52,north,move_forward,53,east) transition(1,12,53,east,move_forward,43,north) transition(1,13,43,north,turn_left,42,west) transition(1,14,42,west,move_forward,41,west) transition(1,15,41,west,move_forward,31,north) transition(1,16,31,north,move_forward,21,north) transition(2,0,68,north,move_forward,58,north) transition(2,1,58,north,move_forward,57,west) transition(2,2,57,west,move_forward,56,west) transition(2,3,56,west,move_forward,46,north) transition(2,4,46,north,turn_left,45,west) transition(2,5,45,west,move_forward,35,north) transition(2,6,35,north,move_forward,25,north) </pre>	(Solving: 4.52s 1st Model: 1.24s Unsat: 3.28s)
	COULDN'T ENFORCE A CRASH	
	<pre> action(1,0,move_forward) action(1,1,move_forward) action(1,2,turn_left) action(1,3,move_forward) action(1,4,move_forward) action(1,5,move_forward) action(1,6,move_forward) action(1,7,move_forward) action(1,8,move_forward) action(1,9,move_forward) action(1,10,move_forward) action(1,11,move_forward) action(1,12,move_forward) action(1,13,turn_left) action(1,14,move_forward) action(1,15,move_forward) action(1,16,move_forward) action(2,0,move_forward) action(2,1,move_forward) action(2,2,move_forward) action(2,3,turn_left) action(2,4,move_forward) action(2,5,move_forward) action(2,6,move_forward) action(2,7,move_forward) </pre>	Yes* *I suspect that here I've run into the problem of two trains 'passing through each other'. I need to diagnose if it's really the case - and if it is, fix it with constraint (from afar, it doesn't seem difficult to prohibit with a constraint; IDEA: test this on a 1d (odd?) environment we two trains are just sent towards each other.

CONCERN: do two opposite trains just pass through each other?

Possible 'Ghost Train' Problem

Let's find this out in a simple 1d environment:

	0	1	2	3	4	5	6
0	1025	1025	1025	1025	1025	1025	1025

"Ghost Train" test env

Let's encode it: **Done.**

01.06.25

Let's run 1 train to test the 1d environment we've just created. **Empty set.** Why? As usual, **the bug is very simple** and is a basic human error: I forgot to indicate the trains for end and start predicates. **Fixed. Works.** Implementing the second, opposite train.

EXPECTATION: the trains arrive at their respective destinations successfully, which is evidence of the 'ghost train' problem. **FALSE:** the encoding is unsatisfiable. To be on the safe side, let's also test the second train separately: ...

Let's also try on the ODD length environment - my initial assumption of the odd environment yielding more to the ghost train problem could be wrong.

INTUITION: the ghost train problem will appear on an EVEN path.

Extending the test environment by 1 cell to make it **odd: Done.** Testing: surprisingly, the answer is still 'unsatisfiable' - which is perfectly good, since we have fewer unsolved problems.

Let's still do more and more diverse testing for the Ghost Train problem (I should test the more complex deterministic environments for the ghost train problem (the ones that I already have - just put the opposite train)):

Test	Result (failure is positive)
1d straight line even	Unsatisfiable
1d straight line odd	Unsatisfiable

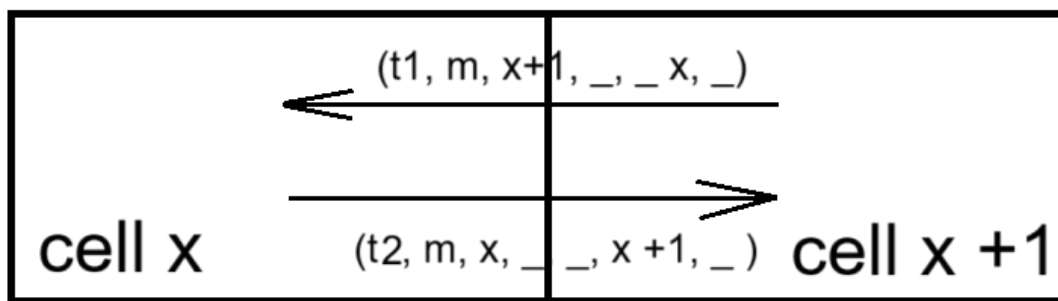
NO, THE GHOST PROBLEM IS HERE:

```
action(0,0,move_forward) action(0,1,move_forward) action(0,2,move_forward)
action(0,3,move_forward) action(0,4,move_forward) action(0,5,move_forward)
action(0,6,move_forward) action(1,0,move_forward) action(1,1,move_forward)
action(1,2,move_forward) action(1,3,move_forward) action(1,4,move_forward)
action(1,5,move_forward) action(1,6,move_forward)
```

OR:

```
transition(0,0,0,east,move_forward,1,east)
transition(0,1,1,east,move_forward,2,east)
transition(0,2,2,east,move_forward,3,east)
transition(0,3,3,east,move_forward,4,east)
transition(0,4,4,east,move_forward,5,east)
transition(0,5,5,east,move_forward,6,east)
transition(0,6,6,east,move_forward,7,east)
transition(1,0,7,west,move_forward,6,west)
transition(1,1,6,west,move_forward,5,west)
transition(1,2,5,west,move_forward,4,west)
transition(1,3,4,west,move_forward,3,west)
transition(1,4,3,west,move_forward,2,west)
transition(1,5,2,west,move_forward,1,west)
transition(1,6,1,west,move_forward,0,west)
```

To understand the required constraint, I must identify the transition atoms that create the problem. Graphically, the impossible transitions look like this (could still be wrong):



Let's express it as a constraint:

Constraint Idea	ASP Phrasing	Code
Two opposite trains must not pass through each other.	There must be no answer set with two 'transition' atoms where: The trains are different, the moment of time is the same, the directions are opposite (might be redundant and/or over-restricting since we can enforce through coordinates alone; action can also be ignored).	<pre>:- transition(TRAIN_1, MOMENT, OUT_DIR, _, _, IN_DIR), transition(TRAIN_2, MOMENT, IN_DIR, _, _, OUT_DIR, _).</pre> Fixing: <pre>:- transition(TRAIN_1, MOMENT, _, IN_POS, _ _, OUT_POS, _), transition(TRAIN_2, MOMENT, _, OUT_POS, _ _, IN_POS, _).</pre>

Testing the constraint: **fails** (there is still an answer). Mistake: not directions - but positions!
Fixed. Result: still **fails**. Analysing: again a **very dumb mistake** in positioning attributes inside the 'transition' atom when describing a constraint. Fixing: **done**. Testing: **success** (unsatisfiable).

CONCLUSION (once more): if an encoding fails, always look first for the dumbest, human-incurred mistakes, not global-scope architectural shortcomings.

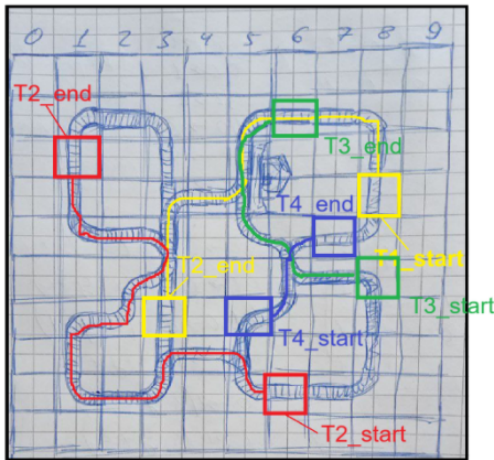
Negative testing of the constraint (checking if doesn't over prohibit):

Environment	Result
1d line odd 1 train	Passes
1d line odd 1 train opposite	Passes
1d line odd 2 trains	Passes (unsatisfiable)
Straight line	Passes
Γ-turn	Fails (dumb mistake: forgot to convert to new coordinates) Fixing: Passes

Π-turn	Passes
Bagel	Passes
Trap switch	Passes
Caduceus	Passes
Bowel	Passes
Single Switch	Passes
Ten by ten	Passes* Passes (2 trains) Passes (3 trains, waiting): (Solving: 1.80s 1st Model: 0.87s Unsat: 0.91s)

*but it's just one train for now and I want to experiment by adding trains; let's put 2 again; still fast for 3 trains.

Add more trains or go to a bigger environment? Let's explore if it's addable for 10x10. It already looks crammed, but I can add one more with a very short track:



Running the 4th added trains. Runs correctly. Adding more 1 by 1:

All run without problems, the new train doesn't require any waiting. Let's try to modify it in a way that waiting is necessary. Experimenting: it's a bit unclear what happens at the diverging track, but we'll see later.

Let's run all 4 trains again to check the computation speed:

(Solving: 8.23s 1st Model: 1.21s Unsat: 0.22s)

Not bad for 4 trains.

Let's go on. Let's jump right to Kep's environments, but switch off trains there and then switch on one by one. As it's written in the description, there are actually **just 2 agents** - must not be too long. **INTUITION: the size of the environment must not be as important, since many of the tracks are just deterministic action - building on top of this intuition, it's natural to come to 'vector' (graph) representation of environments, where only turns / junctions matter.**

Running a 30x30 for 1 train. It seems like there are too many degrees of freedom - the number of answers exceeds 2000. Let's then turn the second train on - the intuition is that it can reduce the number of possible answers and thus make the model 'converge' faster.

It seems that finally the time has come to use 'minimize'. But what can I minimize? The number of 'waiting' atoms?

Let's first use the naive way and simply limit the number of answer sets - to 10, for instance:

(Solving: 2.72s 1st Model: 2.46s Unsat: 0.00s)

Let's try on other/bigger instances - if necessary, we'll handle 'minimize' tomorrow at the lesson:

...

